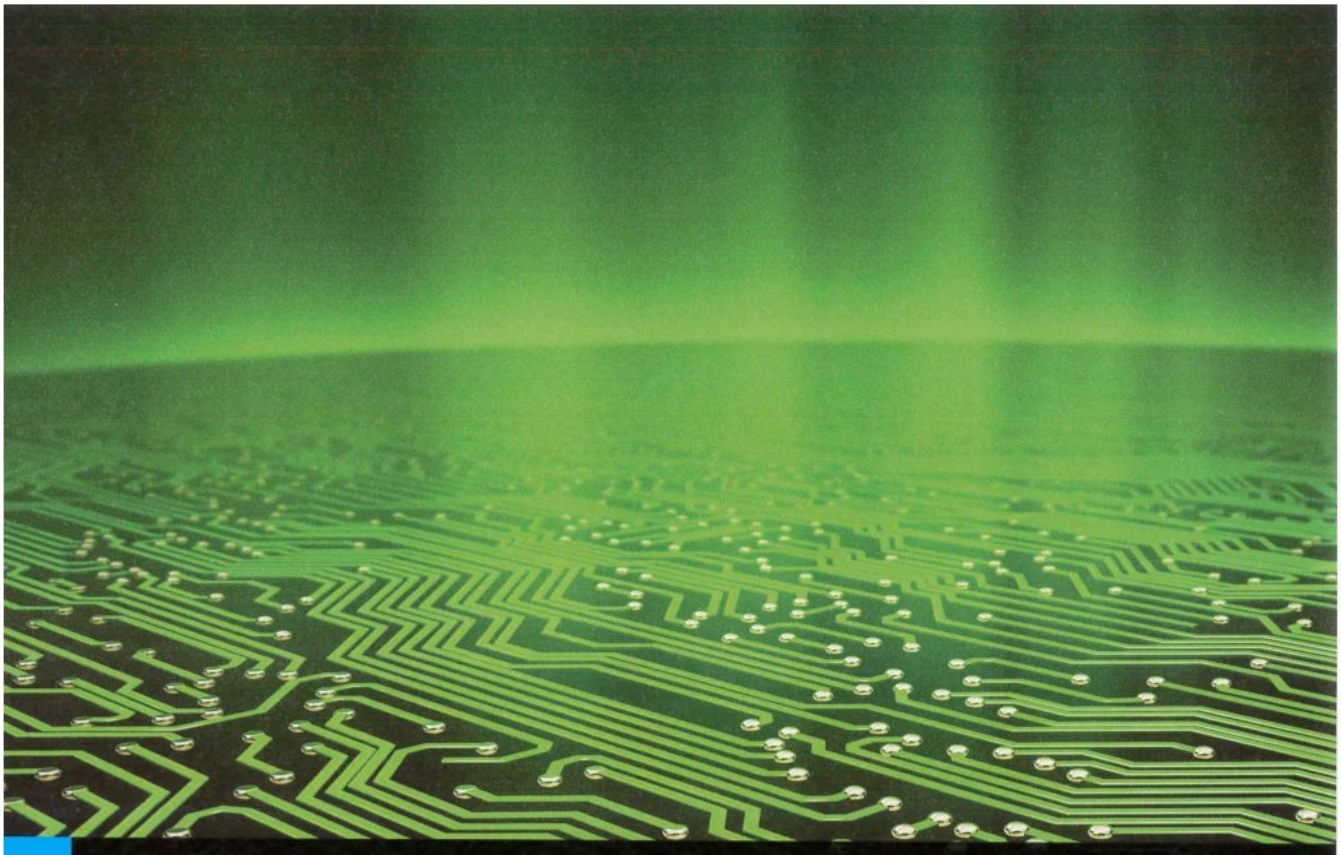
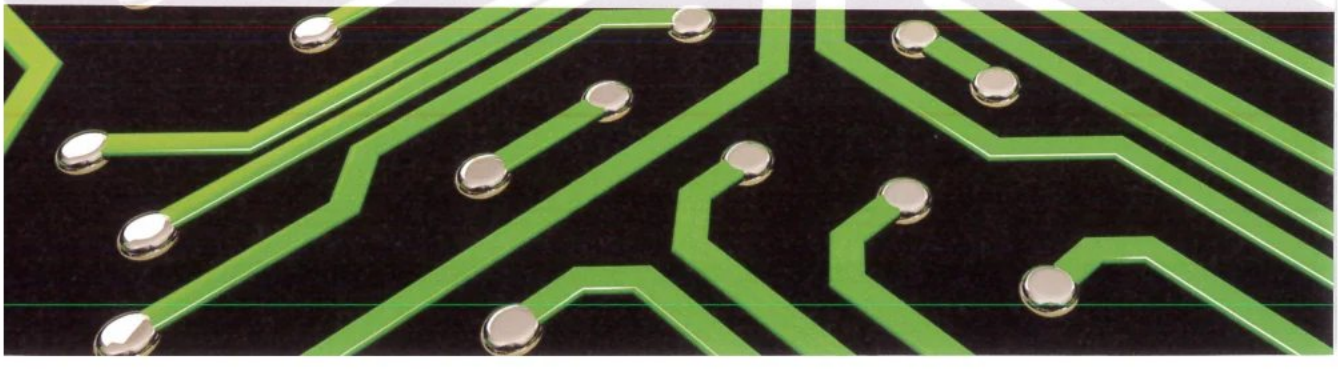


Chapter 18:
Hardware and virtual machines

Learning objectives

By the end of this chapter you should be able to:

- show understanding of Reduced Instruction Set Computers (RISC) and Complex Instruction Set Computers (CISC) processors
- show understanding of the importance/use of pipelining and registers in RISC processors
- show awareness of the four basic computer architectures: SISD, SIMD, MISD, MIMD
- show understanding of the characteristics of massively parallel computers
- show understanding of the concept of a virtual machine.



18.01 The control unit

While a program is being executed, the CPU is receiving a sequence of machine-code instructions. It is the responsibility of the control unit within the CPU to ensure that each machine instruction is handled correctly. There are two methods by which a control unit can be designed to allow it to perform its function.

One method is for the control unit to be constructed as a logic circuit. This is called the hard-wired solution. The machine-code instructions are handled directly by hardware.

The alternative method is for the control unit to use microprogramming. In this approach, the control unit contains a ROM component that stores the microinstructions or microcode for microprogramming, often referred to as firmware. In the next section we will see what might influence the choice of design of the control unit.

18.02 CISC and RISC processors

A processor will have an architecture which refers to its physical construction. However, a processor will also have what is termed an 'instruction set architecture'.

This is concerned with:

- the instruction set
- the instruction format
- the addressing modes
- the registers accessible by instructions.

The choice of the instruction set is the main factor in distinguishing one instruction set architecture from another.

In the early days of computing a significant factor in choosing the instruction set architecture for a computer was that it should make the writing of a compiler for a high-level language easier. At that time the term did not exist, but we would now refer to this as being the architecture for a **Complex Instruction Set Computer (CISC)** .

This philosophy began to be challenged in the late 1970s. It was argued that using a **Reduced Instruction Set Computer (RISC)** would be a better approach. Table 18.01 contains a number of features that distinguish RISC from CISC.



KEY TERMS

Complex Instruction Set Computer (CISC): a single instruction can be more complex and involve more loading of data from memory

Reduced Instruction Set Computer (RISC): a single instruction is simpler, requiring minimal loading of data from memory

RISC	CISC
Fewer instructions	More instructions
Simpler instructions	More complex instructions
Small number of instruction formats	Many instruction formats
Single-cycle instructions whenever possible	Multi-cycle instructions
Fixed-length instructions	Variable-length instructions
Only load and store instructions to address memory	Many types of instructions to address memory
Fewer addressing modes	More addressing modes
Multiple register sets	Fewer registers
Hard-wired control unit	Microprogrammed control unit
Pipelining easier	Pipelining more difficult

Table 18.01 Comparison of RISC with CISC

The following are some points to note.

- For RISC the term 'reduced' affects more than just the number of instructions.
- A reduction in the number of instructions is not the major driving force for the use of RISC.
- The reduction in the complexity of the instructions is a key feature of RISC.
- The typical CISC architecture contains many specialised instructions.
- The specialised instructions are designed to match the requirement of a high-level programming language.
- The specialised instructions require multiple memory accesses which are very slow compared with register accesses.
- The simplicity of the instructions for a RISC processor allows data to be stored in registers and manipulated in them with no resource to memory access other than that necessary for initial loading and possible final storing.
- The simplicity of RISC instructions makes it easier to use hard-wiring inside the control unit.
- The complexity of many of the CISC instructions makes hard-wiring much more difficult so microprogramming is the norm.

Extension Question 18.01

Can you find out whether the processors in any systems you are using are described as RISC or CISC?

Pipelining

One of the major driving forces for creating RISC processors was the opportunity they would provide for efficient **pipelining**. Pipelining is a form of parallelism applied specifically to instruction execution. Other forms of parallelism are discussed in Section 18.03.



KEY TERM

Pipelining: instruction-level parallelism

The underlying principle of pipelining is that the fetch–decode–execute cycle described in Chapter 5 (Section 5.06) can be separated into a number of stages. One possibility is a five-stage model consisting of:

		Clock cycles						
		1	2	3	4	5	6	7
Processor units	Instruction fetch (IF)	1.1	2.1	3.1	4.1	5.1	6.1	7.1
	Instruction decode (ID)		1.2	2.2	3.2	4.2	5.2	6.2
	Operand fetch (OF)			1.3	2.3	3.3	4.3	5.3
	Instruction execute (IE)				1.4	2.4	3.4	4.4
	Result write back (WB)					1.5	2.5	3.5

Figure 18.01 Pipelining for five-stage instruction handling

Figure 18.01 shows how pipelining would work with this five-stage breakdown of instruction handling. For pipelining to be implemented, the construction of the processor must have five independent units, with each handling one of the five stages identified. This explains the need for a RISC processor to have many register sets; each processor unit must have access to its own set of registers. Figure 18.01 uses the representation 1.1, 1.2 and so on to define the instruction and the stage of the instruction. Initially only the first stage of the first instruction has entered the pipeline. At clock cycle 6 the first instruction has left the pipeline, the last stage of instruction 2 is being handled and instruction 6 has just entered.

Once under way, the pipeline is handling five stages of five individual instructions. In particular, at each clock cycle the complete processing of one instruction has finished. Without the pipeline the processing time would be five times longer.

One issue with a pipelined processor is interrupt handling. The discussion in Chapter 5 (Section 5.08) referred to a processor with instructions handled sequentially. This approach where a check for any interrupt is made following the execution of an instruction is applicable to a CISC processor. It would also be applicable to a RISC processor if there were no pipelining. However, this is an unlikely circumstance. In the pipelined system described above there will be five instructions in the pipeline when an interrupt occurs. One option for handling the interrupt is to erase the pipeline contents for the latest four instructions to have entered. Then the normal interrupt-handling routine can be applied to the remaining instruction. The other option is to construct the individual units in the processor with individual program counter registers. This option allows current data to be stored for all of the instructions in the pipeline while the interrupt is handled.

Discussion Point:

Consider the two consecutive instructions:

```
ADD R1, R2, R3

ADD R5, R1, R4
```

These are typical three-register instructions favoured for RISC. The first adds the contents of registers R2 and R3 and stores the result in R1. The next instruction is similar but uses